

# BÁO CÁO BÀI TẬP LỚN

## HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

### 1. Giới thiệu mục tiêu và phạm vi thực hiện

#### 1.1. Mục tiêu:

- Xây dựng một hệ thống đấu giá trực tuyến hoạt động theo mô hình Client - Server, đảm bảo tính nhất quán dữ liệu, cập nhật trạng thái thời gian thực và xử lý đồng thời khi có nhiều người dùng tham gia đấu giá cùng lúc.
- Áp dụng các kiến thức cốt lõi về lập trình hướng đối tượng (OOP) bằng Java, lập trình mạng (Socket), lập trình đa luồng (Multi-threading) và các mẫu thiết kế (Design Patterns).

#### 1.2. Phạm vi thực hiện:

- Phạm vi hệ thống: Quản lý cơ sở dữ liệu, điều phối các phiên đấu giá, xử lý đa luồng cho các kết nối Client.
- Đối tượng sử dụng: Admin (Quản trị hệ thống), Seller (Người bán đăng tài sản), Bidder (Người mua tham gia đấu giá).
- Danh mục sản phẩm: Đồ vật.
- Giới hạn kỹ thuật: Hệ thống chạy trong mạng nội bộ (Local Network) thông qua kết nối TCP/IP Socket; giao diện đồ họa xây dựng bằng JavaFX.
- Common Library: Vùng mã nguồn chung định nghĩa các mô hình dữ liệu (Models) và giao tiếp mạng.

### 2. Kiến trúc tổng thể hệ thống và mô tả

#### 2.1. Sơ đồ kiến trúc phân tán Client-Server:

[ JavaFX Client Application ] <--- (TCP Socket / Object Streams) ---> [ Multi-threaded Server ]

|  
(MVC Architecture)

|  
(Service / DAO)

|  
[ FXML UI + Controllers ]

|  
[ Database (MySQL/PostgreSQL) ]

## 2.2. Mô tả thành phần kiến trúc hệ thống:

- Hệ thống sử dụng cấu trúc **Maven Multi-Module** nhằm chia nhỏ trách nhiệm và tăng khả năng tái sử dụng mã nguồn:
  - **auction\_common**: Chứa các đối tượng dùng chung giữa Client và Server. Giúp đồng bộ hóa dữ liệu thông qua cơ chế Tuần tự hóa đối tượng (Object Serialization) khi truyền tải qua môi trường mạng TCP/IP.
  - **auction\_server**: Hoạt động theo cơ chế Multi-threaded. Lớp AuctionServer mở cổng Socket lắng nghe liên tục. Mỗi khi có Client kết nối, một luồng riêng biệt ClientHandler được khởi tạo để phục vụ độc lập, giúp hệ thống không bị nghẽn (Non-blocking cho các client khác).
  - **auction\_client**: Áp dụng mô hình MVC (Model-View-Controller).
    - View: Các file .fxml kết hợp với style.css để định hình giao diện.
    - Controller: Các lớp điều khiển nhận sự kiện từ UI, gửi yêu cầu tới NetworkClientService để tương tác với Server.

## 3. Chức năng

### 3.1. Cấu trúc cây kế thừa đối tượng hệ thống (Domain Core)

- Hệ thống quản lý thực thể dựa trên mô hình hướng đối tượng phân cấp rõ ràng trong module **auction\_common**:
- Hệ thống người dùng: User (Abstract Class/Superclass) -> Kế thừa bởi: Admin, Seller, Bidder.

### 3.2. Chứng minh việc áp dụng 4 tính chất OOP trong mã nguồn

- Tính đóng gói (Encapsulation): Tất cả các thuộc tính trong các lớp dữ liệu (như Bid, Auction, User) đều được khai báo với phạm vi truy cập private. Trạng thái của đối tượng chỉ được thay đổi hoặc truy xuất thông qua các hàm công khai getter và setter đi kèm với logic kiểm tra tính hợp lệ.
- Tính kế thừa (Inheritance): Giảm thiểu trùng lặp mã nguồn. Lớp cha User chứa các thuộc tính chung như id, username, password, email. Các lớp con (Bidder, Seller) chỉ cần định nghĩa thêm các thuộc tính đặc trưng riêng biệt.
- Tính trừu tượng (Abstraction): Thể hiện rõ nét qua việc định nghĩa Interface BidObserver. Tầng logic chỉ quan tâm đến hành vi "nhận thông báo khi có lượt đặt giá mới" mà không cần biết chi tiết cách Client render giao diện hay cách Server ghi log.

### 3.3. Các Design Patterns áp dụng trong hệ thống

- Observer Pattern (Mẫu quan sát)
  - Lý do lựa chọn: Khi có một người dùng đặt giá (Bid) mới thành công, tất cả các ứng dụng Client khác đang ở trong phòng đấu giá đó cần phải thấy sự thay đổi ngay lập tức mà không cần bấm tải lại trang (F5).
  - Giải pháp trong mã nguồn: Sử dụng Interface BidObserver đặt tại module common. Phía Server, BidLogger đăng ký lắng nghe để ghi nhật ký hệ thống. Phía Client, BidUIObserver triển khai interface này để lắng nghe gói tin từ luồng mạng và đẩy dữ liệu lên màn hình người dùng.
- Singleton Pattern (Mẫu đơn thể)
  - Lý do lựa chọn: Đảm bảo một tài nguyên hệ thống duy nhất được khởi tạo xuyên suốt vòng đời ứng dụng để tránh lãng phí bộ nhớ và xung đột dữ liệu.
  - Giải pháp trong mã nguồn: Áp dụng cho lớp UserSession ở Client (để lưu trữ thông tin tài khoản đang đăng nhập duy nhất trên ứng dụng) và DatabaseService ở Server (quản lý một kết nối duy nhất tới hệ quản trị cơ sở dữ liệu).

### 3.4. Cơ chế Realtime Update qua mạng Socket TCP

- Luồng mạng NetworkClientService ở Client liên tục chạy ngầm (Background Thread) để đọc dữ liệu từ Server thông qua ObjectInputStream.
- Giải quyết xung đột luồng UI trong JavaFX: Do quy tắc của JavaFX không cho phép luồng ngầm can thiệp giao diện, toàn bộ mã nguồn cập nhật đồ họa trong BidUIObserver được bọc trong phương thức quản lý luồng của hệ thống:

### 3.5. Hướng giải quyết bài toán Đấu giá đồng thời (Concurrency Handling)

- Nguy cơ hệ thống: Trong cùng một phần nghìn giây (ms), hai người dùng cùng bấm nút đặt giá cho một sản phẩm với cùng một số tiền lớn hơn giá hiện tại. Nếu không xử lý, hiện tượng Lost Update hoặc Race Condition sẽ xảy ra (cả 2 đều được ghi nhận, gây sai lệch luồng tiền).
- Giải pháp áp dụng: 1. Tại tầng nghiệp vụ của Server (BidService), hàm xử lý đặt giá được bao bọc bằng từ khóa kiểm soát luồng synchronized dựa trên định danh của phiên đấu giá: }
- Kết hợp cơ chế khóa bảo toàn (Pessimistic/Optimistic Locking) ở tầng Database (DatabaseService) để đảm bảo tại một thời điểm chỉ có duy nhất một câu lệnh ghi (Write) lượt đấu giá được thực thi thành công.

### 3.6. Chức năng chính đạt được & Tính năng nâng cao

- Quản lý người dùng, sản phẩm & Chức năng đấu giá

- Hướng giải quyết: Sử dụng phân hệ phân quyền rõ ràng. SellerController cho phép đăng sản phẩm, thiết lập giá khởi điểm và thời gian kết thúc. BidderController cho phép tìm kiếm, lọc sản phẩm.
- Tính năng nâng cao: Trục quan hóa lịch sử đấu giá (Bid History Visualization)
- Hướng giải quyết: Tích hợp RealtimeChartController tại giao diện phòng đấu giá (BiddingRoom.fxml). Mỗi khi nhận được tín hiệu có lượt đặt giá mới từ BidObserver, thành phần LineChart của JavaFX sẽ tự động vẽ thêm một điểm dữ liệu mới theo trục thời gian (X: Thời gian, Y: Giá tiền), giúp người tham gia nhìn rõ xu hướng và nhịp độ của phiên đấu giá theo thời gian thực.

### 3.7. Chiến lược xử lý lỗi và ngoại lệ (Exception Handling)

- Hệ thống xây dựng cơ chế bắt lỗi đa tầng nhằm đảm bảo ứng dụng không bị dừng đột ngột (Crash):
- Lỗi mạng kết nối: Đóng gói mã nguồn truyền tải trong khối try-catch bắt các ngoại lệ SocketException, EOFException. Khi Server ngắt kết nối đột ngột, Client sẽ kích hoạt DialogUtils để hiển thị thông báo thân thiện và tự động chuyển hướng người dùng về màn hình Đăng nhập.
- Lỗi dữ liệu đầu vào: Tại màn hình đặt giá, nếu người dùng nhập chữ thay vì nhập số, hệ thống chủ động bắt lỗi NumberFormatException tại BiddingRoomController và cảnh báo trực tiếp trên UI mà không gửi gói tin lỗi về Server.

### 3.8. Công tác kiểm thử mã nguồn bằng JUnit (Unit Test)

- Kịch bản thực hiện: Thành viên D viết các hàm kiểm thử tự động tại thư mục src/test/java tập trung vào tầng logic cốt lõi của BidService và UserService.
- Nội dung kiểm thử: Kiểm tra tính đúng đắn của hàm kiểm duyệt điều kiện đặt giá (Ví dụ: Test case truyền vào giá thấp hơn giá hiện tại phải trả về kết quả false, truyền vào giá hợp lệ phải trả về true).

### 3.9. Thiết lập luồng tích hợp liên tục CI/CD (GitHub Actions)

- Giải pháp cấu hình: Xây dựng tệp cấu hình tự động .github/workflows/maven.yml kích hoạt mỗi khi thành viên trong nhóm thực hiện thao tác push mã nguồn hoặc pull request lên nhánh chính (main).
- Luồng hoạt động tự động: Hệ thống giả lập môi trường Ubuntu  
 $\rightarrow$  Khởi tạo Java Development Kit (JDK 17/21)  $\rightarrow$  Chạy lệnh biên dịch và kiểm tra toàn bộ hệ thống thông qua Maven
- Luồng này đảm bảo mã nguồn của cả 4 thành viên khi gộp lại luôn sạch, không xảy ra xung đột cấu trúc và vượt qua toàn bộ các bài kiểm thử Unit Test trước khi đóng gói sản phẩm.

#### 4. Bảng phân chia công việc nhóm

| Thành viên                      | Vai trò chính           | Công việc cụ thể thực hiện                                                                                                | Mã nguồn phụ trách                                 |
|---------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Nguyễn Đức Lân<br>(Trưởng nhóm) | Database Engineer       | Thiết kế Cơ sở dữ liệu, xây dựng các lớp Entity/Model kế thừa, lập trình tầng DatabaseService.                            | auction_server<br>(database)<br><br>auction_common |
| Nguyễn Tiến Đạt                 | Backend Developer       | Thiết kế kiến trúc Server, lập trình đa luồng Socket, giải quyết đấu giá đồng thời.                                       | auction_server<br>(network, services)              |
| Trần Đức Hùng                   | Frontend UI/UX Designer | Thiết kế giao diện FXML, xây dựng các Controller tương tác, xử lý luồng giao diện thời gian thực, vẽ biểu đồ lịch sử giá. | auction_client                                     |
| Nguyễn Cao Phong                | QA/QC Tester            | Xử lý ngoại lệ toàn hệ thống, viết kịch bản Unit Test (JUnit), cấu hình luồng tự động hóa CI/CD (GitHub Actions).         | Toàn bộ dự án<br><br>(test, workflow)              |